

Vanishing and Exploding Gradients

DATA 443/643 · Week 2, note 2.2

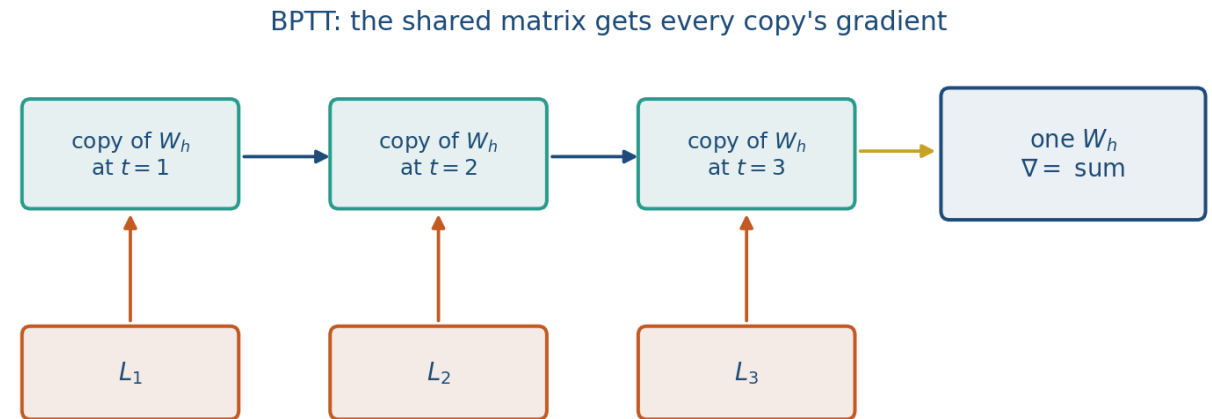
After CS231N L7 (BPTT, singular values) and CS224N L4 (near vs. long-term effects).

Lecture plan

- 1 BPTT: the shared matrix gets a sum
- 2 A product of Jacobians
- 3 Largest |eigenvalue| of W_h
- 4 Clipping helps explosions, not vanishing
- 5 What that does to a language model

Backprop through time

- A repeated weight gets every copy's gradient.
- That sum is BPTT.
- Truncation is for memory. It does not fix vanishing.



A product along the chain

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_T} \frac{\partial h_T}{\partial h_{T-1}} \cdots \frac{\partial h_2}{\partial h_1}$$

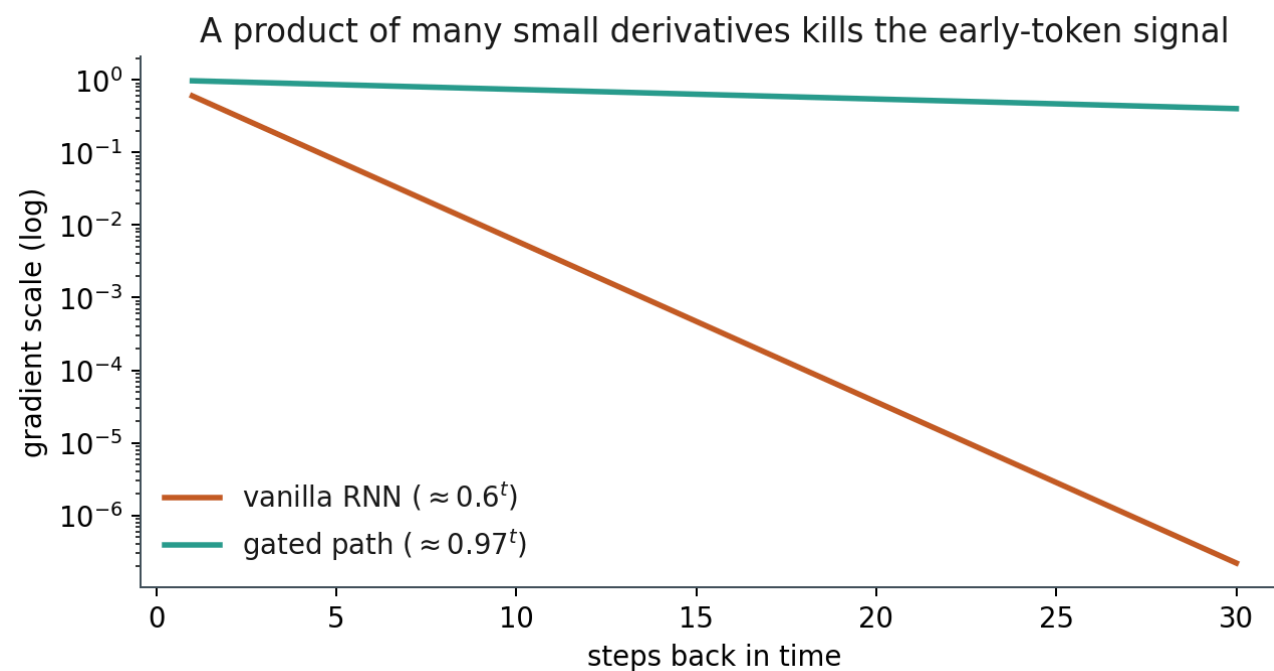
Each factor contains W_h and $\tanh'$.

$|\tanh'| \leq 1$, and usually much smaller.

Twenty factors of **0.5** is already 10^{-6} .

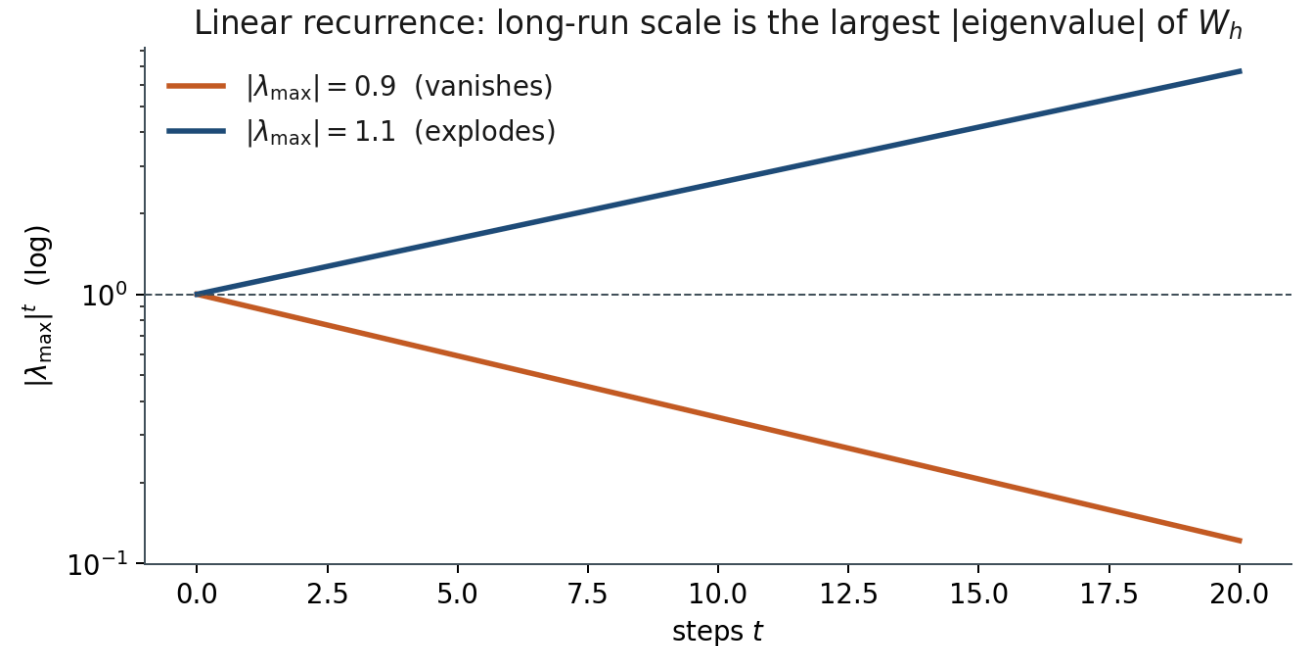
The early token's learning signal dies

- Forward state can still hold a faint trace.
- The gradient does not come back.
- Gates (2.3) keep a path near 1.



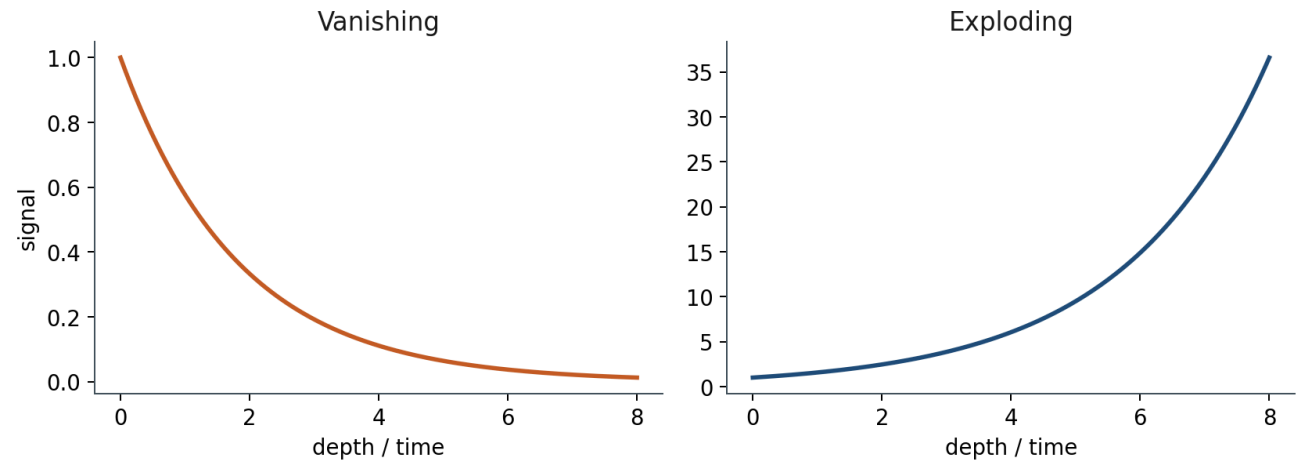
Linear recurrence is an eigenvalue story

- Scale like $|\lambda_{\max}|^t$.
- Below 1: vanish. Above 1: explode.
- Vanilla nets do not learn a unitary W_h .



Two failure modes

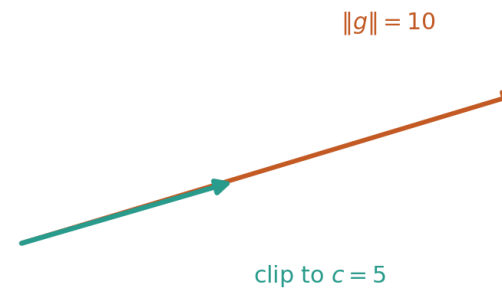
- Vanishing: early tokens do not train.
- Exploding: NaNs, then you restart.
- Same product, opposite disaster.



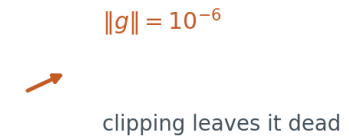
Clipping is a shorter step, not a resurrection

- If $\|g\| > c$, replace g by $c g / \|g\|$.
- Same direction, smaller step.
- A vanished vector is already near 0.

Exploding: same direction, shorter step



Vanishing: there is nothing to rescale



What the product does to next-token training

Near effects

The last few words still move the weights. The model learns local fluency.

Long-term effects

Tickets at step 7 never trains the last-word NLL. At test time the net cannot use that binding.

What to take from this lecture

Sum copies

BPTT is the chain rule on a shared W_h .

Clip explosions

Pascanu: rescale a huge g. Do not claim it fixes vanishing.

Measure Lab 2

If $T=40$ is chance, the product is the bug, not PyTorch.